

Uninformed Search in Python – Part 2

4. Iterative deepening depth-first search (IDDFS)

Iterative deepening depth-first search is a combination of depth-first search and breadth-first search. IDDFS find the best depth limit by gradually adding the limit until the defined goal state is reached. Thus, it is a search algorithm that uses the combined power of the BFS and DFS Algorithm. It is iterative in nature. It searches for the best depth in each iteration. It performs the Algorithm until it reaches the goal node. The algorithm is set to search until a certain depth and the depth keeps increasing at every iteration until it reaches the goal state.

Description of the Algorithm

The basic principle of the algorithm is to start with a start node, and then look at the first child of this node. It then looks at the first child of that node (grandchild of the start node) and so on, until a node has no more children (we've reached a leaf node). It then goes up one level and looks at the next child. If there are no more children, it goes up one more level, and so on, until it finds more children or reaches the start node. If hasn't found the goal node after returning from the last child of the start node, the goal node cannot be found, since by then all nodes have been traversed.

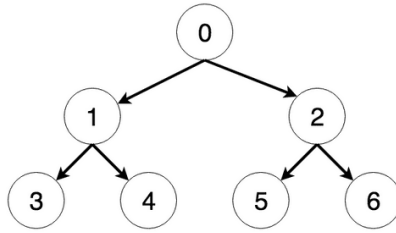
So far this has been describing Depth-First Search (DFS). Iterative deepening adds to this, that the algorithm not only returns one layer up the tree when the node has no more children to visit, but also when a previously specified maximum depth has been reached. Also, if we return to the start node, we increase the maximum depth and start the search all over, until we've visited all leaf nodes (bottom nodes) and increasing the maximum depth won't lead to us visiting more nodes.

Specifically, these are the steps:

1. For each child of the current node
2. If it is the target node, return
3. If the current maximum depth is reached, return
4. Set the current node to this node and go back to 1.
5. After having gone through all children, go to the next child of the parent (the next sibling)
6. After having gone through all children of the start node, increase the maximum depth and go back to 1.
7. If we have reached all leaf (bottom) nodes, the goal node doesn't exist.

Example of the Algorithm

Consider the following tree:



The steps the algorithm performs on this tree if given node 0 as a starting point, in order, are:

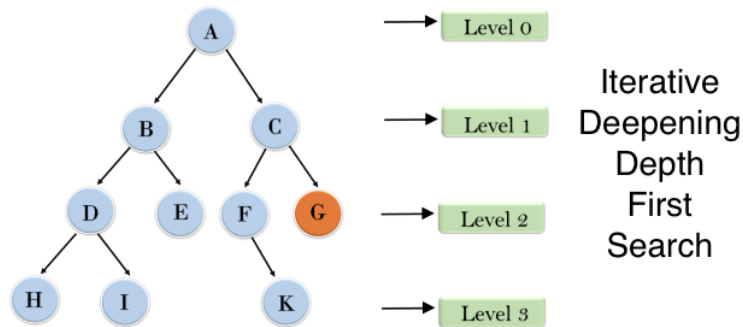
1. Visiting Node 0
2. Visiting Node 1
3. Current maximum depth reached, returning...
4. Visiting Node 2
5. Current maximum depth reached, returning...
6. Increasing depth to 2
7. Visiting Node 0
8. Visiting Node 1
9. Visiting Node 3
10. Current maximum depth reached, returning...
11. Visiting Node 4
12. Current maximum depth reached, returning...
13. Visiting Node 2
14. Visiting Node 5
15. Current maximum depth reached, returning...
16. Visiting Node 6
17. Found the node we're looking for, returning...

Runtime of the Algorithm

If we double the maximum depth each time we need to go deeper, the runtime complexity of Iterative Deepening Depth-First Search (ID-DFS) is the same as regular Depth-First Search (DFS), since all previous depths added up will have the same runtime as the current depth ($1/2 + 1/4 + 1/8 + \dots < 1$). The runtime of regular Depth-First Search (DFS) is $O(|N|)$ ($|N|$ = number of Nodes in the tree), since every node is traversed at most once. The number of nodes is equal to b^d , where b is the branching factor and d is the depth, so the runtime can be rewritten as $O(b^d)$.

Space of the Algorithm

The space complexity of Iterative Deepening Depth-First Search (ID-DFS) is the same as regular Depth-First Search (DFS), which is, if we exclude the tree itself, $O(d)$, with d being the depth, which is also the size of the call stack at maximum depth. If we include the tree, the space complexity is the same as the runtime complexity, as each node needs to be saved.



In the above figure let's consider the goal node to be G and the start state to be A. We perform our IDDFS from node A. In the first iteration, it traverses only node A at level 0. Since the goal is not reached, we expand our nodes, go to the next level i.e., 1 and move to the next iteration. Then in the next iteration, we traverse the node A, B, and C. Even in this iteration our goal state is not reached so we expand the node to the next level i.e., 2, and the nodes are traversed from the start node or the previous iteration and expand the nodes A, B, C, D, E, F, G. Even though the goal node is traversed we go through for the next iteration and the remaining nodes A, B, D, H, I, E, C, F, K, G (BFS & DFS) too are explored, and we find the goal state in this iteration.

Implementation in Python

The following is an implementation of IDDFS in Python. First, I'm including a template (pseudocode) and then the concrete implementation in Python.

```

def iterative_deepening_dfs(start, target):
    """
    Implementation of iterative deepening DFS (depth-first search) algorithm to find
    the shortest path from a start to a target node..

    Given a start node, this returns the node in the tree below the start node with the
    target value (or null if it doesn't exist)

    Runs in  $O(n)$ , where  $n$  is the number of nodes in the tree, or  $O(b^d)$ , where  $b$  is the
    branching factor and  $d$  is the depth.

    :param start: the node to start the search from
    :param target: the value to search for
    :return: The node containing the target value or null if it doesn't exist.
    """

    # Start by doing DFS with a depth of 1, keep doubling depth until we reach the
    # "bottom" of the tree or find the node we're searching for
    depth = 1
    bottom_reached = False # Variable to keep track if we have reached the bottom of
    the tree
    while not bottom_reached:
        # One of the "end nodes" of the search with this depth has to still have
        children and set this to False again
  
```

```

    result, bottom_reached = iterative_deepening_dfs_rec(start, target, 0, depth)
    if result is not None:
        # We've found the goal node while doing DFS with this max depth
        return result

    # We haven't found the goal node, but there are still deeper nodes to search
    through
    depth *= 2
    print("Increasing depth to " + str(depth))

    # Bottom reached is True.
    # We haven't found the node and there were no more nodes that still have children
    to explore at a higher depth.
    return None

def iterative_deepening_dfs_rec(node, target, current_depth, max_depth):
    print("Visiting Node " + str(node["value"]))

    if node["value"] == target:
        # We have found the goal node we we're searching for
        print("Found the node we're looking for!")
        return node, True

    if current_depth == max_depth:
        print("Current maximum depth reached, returning...")
        # We have reached the end for this depth...
        if len(node["children"]) > 0:
            # ...but we have not yet reached the bottom of the tree
            return None, False
        else:
            return None, True

    # Recurse with all children
    bottom_reached = True
    for i in range(len(node["children"])):
        result, bottom_reached_rec = iterative_deepening_dfs_rec(node["children"][i],
target, current_depth + 1,
                                                                    max_depth)

        if result is not None:
            # We've found the goal node while going down that child

```

```

    return result, True

    bottom_reached = bottom_reached and bottom_reached_rec

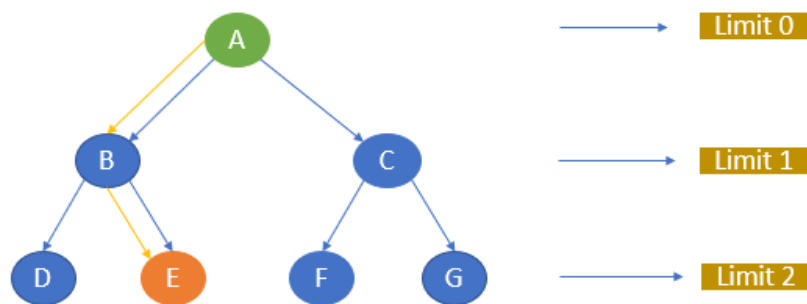
# We've gone through all children and not found the goal node
return None, bottom_reached

```

To explain IDDFS, let use the same example tree.

Consider, A as the start node and E as the goal node. Let the maximum depth be 2.

The algorithm starts with A and goes to the next level and searches for E. If not found, it goes to the next level and finds E.



The path of traversal is A → B → E

Let's try to implement this with Python.

```

graph = {
    'A' : ['B','C'],
    'B' : ['D', 'E'],
    'C' : ['F', 'G'],
    'D' : [],
    'E' : [],
    'F' : [],
    'G' : []
}

path = list()

def DFS(currentNode,destination,graph,maxDepth,curList):
    curList.append(currentNode)
    if currentNode==destination:
        return True

```

```

    if maxDepth<=0:
        path.append(curList)
        return False

    for node in graph[currentNode]:
        if DFS(node,destination,graph,maxDepth-1,curList):
            return True
        else:
            curList.pop()

    return False

def iterativeDDFS(currentNode,destination,graph,maxDepth):
    for i in range(maxDepth):
        curList = list()
        if DFS(currentNode,destination,graph,i,curList):
            return True
    return False

if not iterativeDDFS('A','E',graph,3):
    print("Path is not available")
else:
    print("Path exists")
    print(path.pop())

```

The path generated is as follows.

```

Path exists
['A', 'B', 'E']

```

Advantages of IDDFS

- IDDFS has the advantages of both BFS and DFS. It offers fast search and uses memory efficiently.

Disadvantages of IDDFS

- The main drawback of IDDFS is that it repeats all the work from the previous phase. It does all the works of the previous stage again and again.

Conclusions

Complete: Yes (by limiting the depth)

Time Complexity: $O(b^d)$

Space complexity: $O(bd)$

Optimal: Yes (if step cost is uniform)

Systematic: It's not systematic.

5. Bidirectional search (BS)

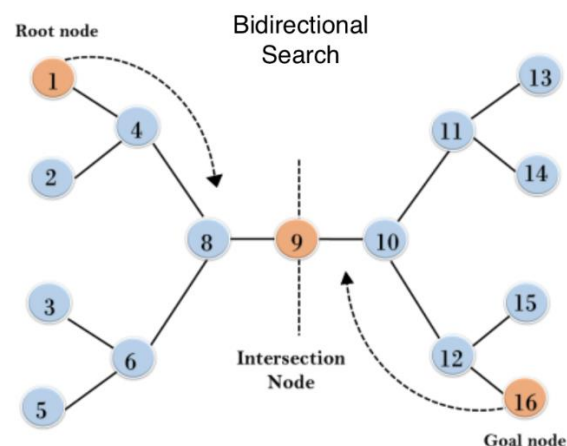
Before moving into bidirectional search let's first understand a few terms.

Forward Search: Looking in-front of the end from start.

Backward Search: Looking from end to the start back-wards.

So Bidirectional Search, as the name suggests, is a combination of forwarding and backward search. Basically, if the average branching factor going out of node / fan-out, if fan-out is less, prefer forward search. Else if the average branching factor is going into a node/fan in is less (i.e., fan-out is more), prefer backward search. We must traverse the tree from the start node and the goal node and wherever they meet the path from the start node to the goal through the intersection is the optimal solution. The BS Algorithm is applicable when generating predecessors is easy in both forward and backward directions and there exist only 1 or fewer goal states.

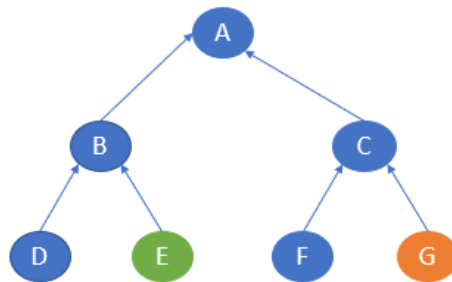
The bidirectional search algorithm is completely different from all other search strategies. It executes two simultaneous searches called forward-search and backwards-search and reaches the goal state. Here, the graph is divided into two smaller sub-graphs. In one graph, the search is started from the initial start state and in the other graph, the search is started from the goal state. When these two nodes intersect each other, the search will be terminated.



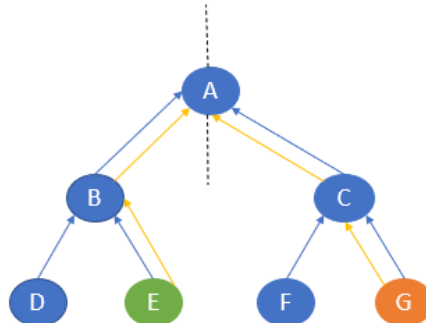
This figure provides a clear-cut idea of how BS is executed. We have node 1 as the start/root node and node 16 as the goal node. The algorithm divides the search tree into two sub-trees. So, from start node 1, we do a forward search and at the same time, we do a backward search from goal node 16. The forward search

traverses' nodes 1, 4, 8, and 9 whereas the backward search traverses through nodes 16, 12, 10, and 9. We see that both forward and backward search meets at node 9 called the intersection node. So, the total path traced by forwarding search and the path traced by backward search is the optimal solution. This is how the BS Algorithm is implemented.

Bidirectional search requires both start and goal start to be well defined and the branching factor to be the same in the two directions. Consider the below graph.



Here, the start state is E and the goal state is G. In one sub-graph, the search starts from E and in the other, the search starts from G. E will go to B and then A. G will go to C and then A. Here, both the traversal meets at A and hence the traversal ends.



The path of traversal is

E → B → A → C → G

Implementation in Python

Let's implement the same in Python.

```
from collections import deque

class Node:
    def __init__(self, val, neighbors=[]):
        self.val = val
        self.neighbors = neighbors
```



```

        self.visited_right = False
        self.visited_left = False
        self.parent_right = None
        self.parent_left = None
def bidirectional_search(s, t):
    def extract_path(node):
        node_copy = node
        path = []
        while node:
            path.append(node.val)
            node = node.parent_right
        path.reverse()
        del path[-1]
        while node_copy:
            path.append(node_copy.val)
            node_copy = node_copy.parent_left
        return path
    q = deque([])
    q.append(s)
    q.append(t)
    s.visited_right = True
    t.visited_left = True
    while len(q) > 0:
        n = q.pop()
        if n.visited_left and n.visited_right:
            return extract_path(n)
        for node in n.neighbors:
            if n.visited_left == True and not node.visited_left:
                node.parent_left = n
                node.visited_left = True
                q.append(node)
            if n.visited_right == True and not node.visited_right:
                node.parent_right = n
                node.visited_right = True

```

```

        q.append(node)

    return False

n0 = Node('A')
n1 = Node('B')
n2 = Node('C')
n3 = Node('D')
n4 = Node('E')
n5 = Node('F')
n6 = Node('G')

n0.neighbors = []
n1.neighbors = [n0]
n2.neighbors = [n0]
n3.neighbors = [n1]
n4.neighbors = [n1]
n5.neighbors = [n2]
n6.neighbors = [n2]

print(bidirectional_search(n4, n6))

```

The path is generated as follows.

```
['E', 'B', 'A', 'C', 'G']
```

The following is another implementation of BS:

```

class adjacent_Node:

    def __init__(self, v):

        self.vertex = v
        self.next = None

class bidirectional_Search:

    def __init__(self, vertices):

        self.vertices = vertices
        self.graph = [None] * self.vertices

        self.source_queue = list()
        self.last_node_queue = list()

```

```

self.source_visited = [False] * self.vertices
self.last_node_visited = [False] * self.vertices

self.source_parent = [None] * self.vertices
self.last_node_parent = [None] * self.vertices

def AddEdge(self, source, last_node):

    node = adjacent_Node(last_node)
    node.next = self.graph[source]
    self.graph[source] = node

    node = adjacent_Node(source)
    node.next = self.graph[last_node]
    self.graph[last_node] = node

def breadth_fs(self, direction = 'forward'):

    if direction == 'forward':

        current = self.source_queue.pop(0)
        connected_node = self.graph[current]

        while connected_node:
            vertex = connected_node.vertex

            if not self.source_visited[vertex]:
                self.source_queue.append(vertex)
                self.source_visited[vertex] = True
                self.source_parent[vertex] = current

            connected_node = connected_node.next
    else:

        current = self.last_node_queue.pop(0)
        connected_node = self.graph[current]

        while connected_node:
            vertex = connected_node.vertex

            if not self.last_node_visited[vertex]:
                self.last_node_queue.append(vertex)
                self.last_node_visited[vertex] = True
                self.last_node_parent[vertex] = current

```

```

        connected_node = connected_node.next

def is_intersecting(self):
    #
    for i in range(self.vertices):
        if (self.source_visited[i] and
            self.last_node_visited[i]):
            return i

    return -1

def path_st(self, intersecting_node,
            source, last_node):

    path = list()
    path.append(intersecting_node)
    i = intersecting_node

    while i != source:
        path.append(self.source_parent[i])
        i = self.source_parent[i]

    path = path[::-1]
    i = intersecting_node

    while i != last_node:
        path.append(self.last_node_parent[i])
        i = self.last_node_parent[i]

    print("*****Path*****")
    path = list(map(str, path))

    print(' '.join(path))

def bidirectional_search(self, source, last_node):

    self.source_queue.append(source)
    self.source_visited[source] = True
    self.source_parent[source] = -1

    self.last_node_queue.append(last_node)
    self.last_node_visited[last_node] = True
    self.last_node_parent[last_node] = -1

```

```

        while self.source_queue and self.last_node_queue:

            self.breadth_fs(direction = 'forward')

            self.breadth_fs(direction = 'backward')

            intersecting_node = self.is_intersecting()

            if intersecting_node != -1:
                print("Path exists between {} and {}".format(source, last_node))
                print("Intersection at : {}".format(intersecting_node))
                self.path_st(intersecting_node,
                             source, last_node)

                exit(0)
            return -1

if __name__ == '__main__':

    n = 17

    source = 1

    last_node = 16

    my_Graph = bidirectional_Search(n)
    my_Graph.AddEdge(1, 4)
    my_Graph.AddEdge(2, 4)
    my_Graph.AddEdge(3, 6)
    my_Graph.AddEdge(5, 6)
    my_Graph.AddEdge(4, 8)
    my_Graph.AddEdge(6, 8)
    my_Graph.AddEdge(8, 9)
    my_Graph.AddEdge(9, 10)
    my_Graph.AddEdge(10, 11)
    my_Graph.AddEdge(11, 13)
    my_Graph.AddEdge(11, 14)
    my_Graph.AddEdge(10, 12)
    my_Graph.AddEdge(12, 15)
    my_Graph.AddEdge(12, 16)

    out = my_Graph.bidirectional_search(source, last_node)

    if out == -1:
        print("No path between {} and {}".format(source, last_node))

```

The path is generated as follows.

```

Path exists between 1 and 16
Intersection at : 9
*****Path*****
1 4 8 9 10 12 16

```

Advantages of bidirectional search

- Since BS uses various techniques like DFS, BFS, DLS, etc, it is efficient and requires less memory. Thus, BS searches the graph fast.
- It requires less memory to complete its action.

Disadvantages of bidirectional search

- The goal state should be pre-defined, so we should know the goal state in advance.
- The graph is quite difficult to implement.

Conclusions

Complete: Yes

Time Complexity: $O(b^{d/2})$

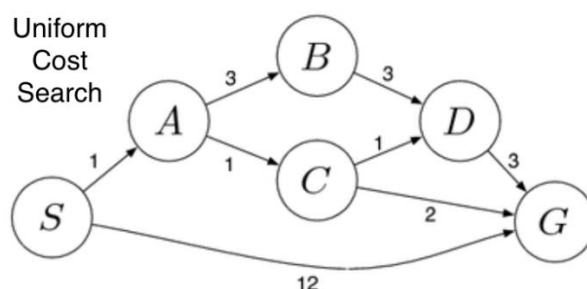
Space complexity: $O(b^{d/2})$

Optimal: Yes (if step cost is uniform in both forward and backward directions)

6. Uniform cost search

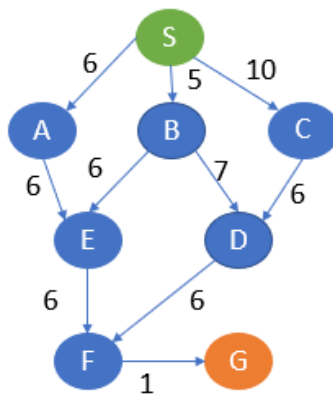
Uniform cost search is considered the best search algorithm for a weighted graph or graph with costs. It searches the graph by giving maximum priority to the lowest cumulative cost. Uniform cost search can be implemented using a priority queue.

This algorithm is mainly used when the step costs are not the same, but we need the optimal solution to the goal state. In such cases, we use Uniform Cost Search to find the goal and the path including the cumulative cost to expand each node from the root node to the goal node. It does not go depth or breadth, it searches for the next node with the lowest cost and in the case of the same path cost, let's consider lexicographical order in our case.



In the above figure consider S to be the start node and G to be the goal state. From node S we look for a node to expand and we have nodes A and G but since it's a uniform cost search it's expanding the node with the lowest step cost, so node A becomes the successor rather than our required goal node G. From A we look at its children nodes B and C. So, since C has the lowest step cost it traverses through node C and then we look at successors of C i.e., D and G since the cost to D is low, we expand along with the node D. Since D has only one child G which is our required goal state, we finally reach the goal state D by implementing UFS Algorithm. If we have traversed this way our total path cost from S to G is just 6 even after traversing through many nodes rather than going to G directly where the cost is 12 and $6 < 12$ (in terms of step cost). But this may not work with all cases.

Consider the below graph where each node has a pre-defined cost.



Here, S is the start node and G is the goal node.

From S, G can be reached in the following ways.

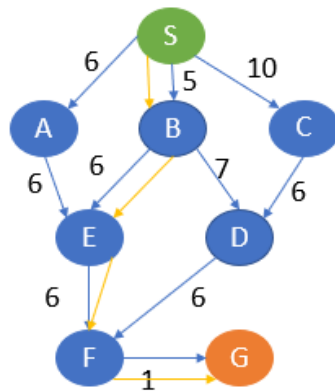
S, A, E, F, G -> 19

S, B, E, F, G -> 18

S, B, D, F, G -> 19

S, C, D, F, G -> 23

Here, the path with the least cost is S, B, E, F, G.



Implementation in Python

Let's implement UCS in Python.

```
graph=[['S','A',6],
        ['S','B',5],
        ['S','C',10],
        ['A','E',6],
        ['B','E',6],
        ['B','D',7],
        ['C','D',6],
        ['E','F',6],
        ['D','F',6],
        ['F','G',1]]

temp = []
temp1 = []
for i in graph:
    temp.append(i[0])
    temp1.append(i[1])
nodes = set(temp).union(set(temp1))

def UCS(graph, costs, open, closed, cur_node):
    if cur_node in open:
        open.remove(cur_node)
    closed.add(cur_node)
    for i in graph:
```



```

    if(i[0] == cur_node and costs[i[0]]+i[2] < costs[i[1]]):
        open.add(i[1])
        costs[i[1]] = costs[i[0]]+i[2]
        path[i[1]] = path[i[0]] + ' -> ' + i[1]
costs[cur_node] = 999999
small = min(costs, key=costs.get)
if small not in closed:
    UCS(graph, costs, open,closed, small)
costs = dict()
temp_cost = dict()
path = dict()
for i in nodes:
    costs[i] = 999999
    path[i] = ' '
open = set()
closed = set()
start_node = input("Enter the Start State: ")
open.add(start_node)
path[start_node] = start_node
costs[start_node] = 0
UCS(graph, costs, open, closed, start_node)
goal_node = input("Enter the Goal State: ")
print("Path with least cost is: ",path[goal_node])

```

The optimal output path is generated.

```

Enter the Start State: S
Enter the Goal State: G
Path with least cost is:  S -> B -> E -> F -> G

```

Advantages of UCS

- Uniform cost search is optimal as the selection of paths is based on the lowest cost.

Disadvantages of UCS

- The algorithm does not consider how many steps it goes to reach the lowest path. It does not care about the number of steps involved in searching and only concerned about path cost. This may result in an infinite loop.

Conclusions

Complete: Yes (if b is finite and costs are stepped costs are zero)

Time Complexity: $O(b^{(c/\epsilon)})$ where, $\epsilon \rightarrow$ is the lowest cost, $c \rightarrow$ optimal cost

Space complexity: $O(b^{(c/\epsilon)})$

Optimal: Yes (even for non-even cost)

Dijkstra's Algorithm

The Dijkstra algorithm is an algorithm used to solve the shortest path problem in a graph. This means that given a number of nodes and the edges between them as well as the “length” of the edges (referred to as “weight”), the Dijkstra algorithm finds the shortest path from the specified start node to all other nodes. Nodes are sometimes referred to as vertices (plural of vertex) - here, we’ll call them nodes.

Description of the Algorithm

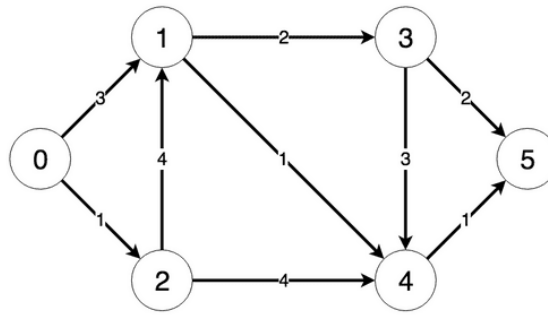
The basic principle behind the Dijkstra algorithm is to iteratively look at the node with the currently smallest distance to the source and update all not yet visited neighbours if the path to it *via* the current node is shorter. In more detail, this leads to the following Steps:

1. Initialize the distance to the starting node as 0 and the distances to all other nodes as infinite
2. Set all nodes to “unvisited”
3. While we haven’t visited all nodes:
 1. Find the node with currently shortest distance from the source (for the first pass, this will be the source node itself)
 2. For all nodes next to it that we haven’t visited yet, check if the currently smallest distance to that neighbour is bigger than if we were to go via the current node
 3. If it is, update the smallest distance of that neighbour to be the distance from the source to the current node plus the distance from the current node to that neighbour

In the end, the array we used to keep track of the currently shortest distance from the source to all other nodes will contain the (final) shortest distances.

Example of the Algorithm

Consider the following graph:



The steps the algorithm performs on this graph if given node 0 as a starting point, in order, are:

1. Visiting node 0
2. Updating distance of node 1 to 3
3. Updating distance of node 2 to 1
4. Visited nodes: 0
5. Currently lowest distances: [0, 3, 1, Infinite, Infinite, Infinite]
6. Visiting node 1 with current distance 1
 - Updating distance of node 3 to 5
 - Visited nodes: 0, 2
 - Currently lowest distances: [0, 3, 1, 5, Infinite, Infinite]
7. Visiting node 3 with current distance 3
 - Updating distance of node 4 to 4
 - Visited nodes: 0, 1, 2
 - Currently lowest distances: [0, 3, 1, 5, 4, Infinite]
8. Visiting node 4 with current distance 4
 - Updating distance of node 5 to 5
 - Visited nodes: 0, 1, 2, 4
 - Currently lowest distances: [0, 3, 1, 5, 4, 5]
9. Visiting node 5 with current distance 5
 - No distances to update
 - Visited nodes: 0, 1, 2, 3, 4
 - Currently lowest distances: [0, 3, 1, 5, 4, 5]
10. Visiting node 5 with current distance 5
 - No distances to update
 - Visited nodes: 0, 1, 2, 3, 4, 5

All nodes visited Final lowest distances: [0, 3, 1, 5, 4, 5]

Runtime of the Algorithm

The runtime complexity of Dijkstra depends on how it is implemented. If a min-heap is used to determine the next node to visit, and adjacency is implemented using adjacency lists, the runtime is $O(|E| + |V|\log|V|)$ ($|V|$ = number of Nodes, $|E|$ = number of Edges). If we simply search all distances to find the node with the lowest distance in each step and use a matrix to look up whether two nodes are adjacent, the runtime complexity increases to $O(|V|^2)$.

Space of the Algorithm

The space complexity of Dijkstra depends on how it is implemented as well and is equal to the runtime complexity.

```
def dijkstra(graph, start):
    """
    Implementation of dijkstra using adjacency matrix.
    This returns an array containing the length of the shortest path from the start
    node to each other node.
    It is only guaranteed to return correct results if there are no negative edges in
    the graph. Positive cycles are fine.
    This has a runtime of  $O(|V|^2)$  ( $|V|$  = number of Nodes), for a faster implementation
    see @see ../fast/Dijkstra.java (using adjacency lists)

    :param graph: an adjacency-matrix-representation of the graph where (x,y) is the
    weight of the edge or 0 if there is no edge.
    :param start: the node to start from.
    :return: an array containing the shortest distances from the given start node to
    each other node
    """
    # This contains the distances from the start node to all other nodes
    distances = [float("inf") for _ in range(len(graph))]

    # This contains whether a node was already visited
    visited = [False for _ in range(len(graph))]

    # The distance from the start node to itself is of course 0
    distances[start] = 0

    # While there are nodes left to visit...
    while True:

        # ... find the node with the currently shortest distance from the start node...
        shortest_distance = float("inf")
        shortest_index = -1
        for i in range(len(graph)):
            # ... by going through all nodes that haven't been visited yet
            if distances[i] < shortest_distance and not visited[i]:
                shortest_distance = distances[i]
                shortest_index = i
```

```

    # print("Visiting node " + str(shortest_index) + " with current distance " +
    str(shortest_distance))

    if shortest_index == -1:
        # There was no node not yet visited --> We are done
        return distances

    # ...then, for all neighboring nodes that haven't been visited yet....
    for i in range(len(graph[shortest_index])):
        # ...if the path over this edge is shorter...
        if graph[shortest_index][i] != 0 and distances[i] >
distances[shortest_index] + graph[shortest_index][i]:
            # ...Save this path as new shortest path.
            distances[i] = distances[shortest_index] + graph[shortest_index][i]
            # print("Updating distance of node " + str(i) + " to " +
str(distances[i]))

    # Lastly, note that we are finished with this node.
    visited[shortest_index] = True
    # print("Visited nodes: " + str(visited))
    # print("Currently lowest distances: " + str(distances))

```

Provided that all of the vertices are reachable from the source vertex; Dijkstra's algorithm can be used to find the shortest distance from the source vertex to all other vertices in a weighted graph. The graph can be directed or undirected, cyclic or acyclic, but the weights on all edges need to be nonnegative.

Thus, as explained above:

1. From each of the unvisited vertices, choose the vertex with the smallest distance and visit it.
2. Then, update the distance for each neighbouring vertex, of the visited vertex, whose current distance is greater than its sum and the weight of the edge between them.
3. Repeat steps 1 and 2 until all the vertices are visited.

Implementation in Python

Let's implement Dijkstra in Python.

```

import sys

# Function to find out which of the unvisited node
# needs to be visited next
def to_be_visited():

```

```

global visited_and_distance

v = -10

# Choosing the vertex with the minimum distance
for index in range(number_of_vertices):
    if visited_and_distance[index][0] == 0 \
        and (v < 0 or visited_and_distance[index][1] <= \
            visited_and_distance[v][1]):
        v = index
return v

# Creating the graph as an adjacency matrix
vertices = [[0, 1, 1, 0],
            [0, 0, 1, 0],
            [0, 0, 0, 1],
            [0, 0, 0, 0]]
edges = [[0, 3, 4, 0],
         [0, 0, 0.5, 0],
         [0, 0, 0, 1],
         [0, 0, 0, 0]]

number_of_vertices = len(vertices[0])

# The first element of the lists inside visited_and_distance
# denotes if the vertex has been visited.
# The second element of the lists inside the visited_and_distance
# denotes the distance from the source.
visited_and_distance = [[0, 0]]
for i in range(number_of_vertices-1):
    visited_and_distance.append([0, sys.maxsize])

for vertex in range(number_of_vertices):
    # Finding the next vertex to be visited.
    to_visit = to_be_visited()
    for neighbor_index in range(number_of_vertices):

```

```

# Calculating the new distance for all unvisited neighbours
# of the chosen vertex.
if vertices[to_visit][neighbor_index] == 1 and \
    visited_and_distance[neighbor_index][0] == 0:
    new_distance = visited_and_distance[to_visit][1] \
        + edges[to_visit][neighbor_index]
    # Updating the distance of the neighbor if its current distance
    # is greater than the distance that has just been calculated
    if visited_and_distance[neighbor_index][1] > new_distance:
        visited_and_distance[neighbor_index][1] = new_distance
# Visiting the vertex found earlier
visited_and_distance[to_visit][0] = 1

i = 0

# Printing out the shortest distance from the source to each vertex
for distance in visited_and_distance:
    print("The shortest distance of ",chr(ord('a') + i),\
        " from the source vertex a is:",distance[1])
    i = i + 1

```

This is the output :

```

The shortest distance of  a  from the source vertex a is: 0
The shortest distance of  b  from the source vertex a is: 3
The shortest distance of  c  from the source vertex a is: 3.5
The shortest distance of  d  from the source vertex a is: 4.5

```

Note. It is clear how Uniform-Cost Search is a variant of Dijkstra's algorithm. Indeed, as we discussed earlier, instead of inserting all vertices into a priority queue, we insert only source, then one by one insert when needed. In every step, we check if the item is already in priority queue (using visited array). If yes, we perform decrease key, else we insert it. This variant of Dijkstra is useful for infinite graphs and those graphs which are too large to represent in the memory. Uniform-Cost Search is mainly used in Artificial Intelligence.

Comparison of uniformed search algorithms - Final Interpretations

The Uninformed Search strategies for searching is a multipurpose strategy that combines the power of unguided search and works in a brute force way. The algorithms of this strategy can be applied in a variety

of problems in computer science as they don't have the information related to state space and target problems.

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Complete	Yes ^a	Yes ^{a,b}	No	No	Yes ^a	Yes ^{a,d}
Time Complexity	$O(b^d)$	$O(b^{(b+c/E)})$	$O(b^m)$	$O(b^l)$	$O(b^d)$	$O(b^{d/2})$
Space Complexity	$O(b^d)$	$O(b^{(b+c/E)})$	$O(bm)$	$O(bl)$	$O(bd)$	$O(b^{d/2})$
Optimal	Yes ^c	Yes	No	No	Yes ^c	Yes ^{c,d}

b -> branching factor

d -> depth of the shallowest solution

m -> max. depth of search tree

l -> depth limit

a -> complete if b is finite

b -> complete if step costs $\geq E$ for $E > 0$

c -> optimal if step costs are same

d -> if both directions use BFS